



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

BÁO CÁO ĐỒ ÁN THỰC HÀNH

SUY LUẬN NGÔN NGỮ TỰ NHIÊN TIẾNG VIỆT TRÊN BỘ DỮ LIỆU ĐỐI KHÁNG ViANLI

Bộ dữ liệu: ViANLI (UIT-NLP)

Môn học:	DEEP LEARNING (AIN501)
Lớp/Nhóm:	MSA37HCM – Đỗ Hoàng Tỷ Phú – 25MS23327
Giảng viên:	Lê Thanh Tùng
Học kỳ:	Summer 2026
Ngày nộp:	15/08/2026

Yêu cầu đồ án và cách sử dụng mẫu báo cáo

Căn cứ học phần

Mẫu báo cáo này được thiết kế theo định hướng học phần AIN501: thực hành xây dựng và huấn luyện CNN, RNN/LSTM và Transformer; sử dụng các kỹ thuật cải thiện mô hình như Dropout, Batch Normalization, khởi tạo Xavier/He; chẩn đoán lỗi hệ thống học máy; áp dụng transfer learning, multi-task learning và thực hiện đồ án thực hành bằng Python/TensorFlow. Nội dung đồ án đồng thời phù hợp với các mô-đun CNN, object detection, RNN, NLP/word embeddings, attention và Transformer.

Yêu cầu bắt buộc

Mỗi bạn chọn **một bộ dữ liệu** và thực hiện tối thiểu **bốn mô hình**:

1. một mô hình thuộc họ **CNN** hoặc CNN-based;
2. một mô hình thuộc họ **RNN/LSTM/GRU** hoặc mô hình xử lý chuỗi tương đương;
3. một mô hình thuộc họ **Transformer**;
4. một **mô hình tham khảo bên ngoài** từ bài báo, GitHub, Kaggle hoặc kho mô hình công khai.

Các mô hình phải được đánh giá trên cùng giao thức dữ liệu và metric phù hợp. Với bài toán mà một họ kiến trúc không phù hợp tự nhiên, nhóm phải giải thích rõ và thực nghiệm mô hình thay thế.

Sản phẩm phải nộp

- ☐ Báo cáo PDF biên dịch từ mẫu này.
- ☐ Mã nguồn chạy được trên Google Colab/Kaggle hoặc máy cục bộ.
- ☐ Tập `requirements.txt` hoặc mô tả môi trường thực nghiệm.

Nguyên tắc so sánh công bằng

1. Sử dụng cùng cách chia train/validation/test hoặc cùng các fold đánh giá.

2. Cố định random seed và báo cáo seed đã dùng.
3. Dùng cùng metric chính; giải thích metric phụ và tiêu chí chọn mô hình tốt nhất.
4. Báo cáo rõ pretrained weights, data augmentation, tokenizer, kích thước đầu vào và ngưỡng dự đoán.
5. Không sử dụng test set để điều chỉnh siêu tham số.
6. Khi dùng code bên ngoài, ghi rõ URL, phiên bản/commit, giấy phép và phần đã chỉnh sửa.

Liên hệ với tiêu chí đánh giá của học phần

Phần đánh giá trong syllabus nhấn mạnh: tính đúng đắn, sáng tạo và giải quyết vấn đề, mức độ hoàn thiện sản phẩm, kỹ năng trình bày, khả năng vận dụng lý thuyết và nộp bài đúng hạn. Vì vậy, báo cáo không chỉ trình bày điểm số tốt nhất mà còn phải trình bày kỹ càng quy trình thực nghiệm, phân tích lỗi có chiều sâu và khả năng tái lập.

Mục lục

Yêu cầu đề án và cách sử dụng mẫu báo cáo	i
Danh mục hình	v
Danh mục bảng	vi
1 Tóm tắt	1
2 Giới thiệu bài toán	2
2.1 Bối cảnh và động lực	2
2.2 Phát biểu bài toán	2
2.3 Mục tiêu và câu hỏi thực nghiệm	3
3 Bộ dữ liệu và tiền xử lý	4
3.1 Nguồn và giấy phép dữ liệu	4
3.2 Khám phá dữ liệu	4
3.2.1 Phân bố nhãn và mốc dưới	5
3.2.2 Độ dài chuỗi và lựa chọn max_length	5
3.2.3 Dò tìm annotation artifact	6
3.2.4 Ba baseline phi-neural	6
3.3 Chia dữ liệu và phòng tránh rò rỉ	7
3.3.1 Kiểm tra rò rỉ dữ liệu	7
3.4 Tiền xử lý và tăng cường dữ liệu	9
3.4.1 Vì sao tách từ tiếng Việt là bước quyết định	9
4 Phương pháp và kiến trúc mô hình	10
4.1 Tổng quan quy trình	10
4.2 Mô hình 1: CNN-based (TextCNN)	10
4.3 Mô hình 2: RNN/LSTM/GRU-based (BiLSTM + attention)	11
4.4 Mô hình 3: Transformer-based (PhoBERT)	12
4.5 Mô hình 4: Mô hình tham khảo bên ngoài (XLM-RoBERTa large)	13
4.5.1 Khẳng định mô hình không được huấn luyện trên tập test của đề tài	14
4.6 Hàm mất mát và chiến lược tối ưu	15
5 Thiết kế thực nghiệm	16
5.1 Môi trường thực nghiệm	16

5.2	Cấu hình huấn luyện	17
5.3	Metric đánh giá	17
5.4	Kế hoạch thí nghiệm và ablation	18
6	Kết quả thực nghiệm	19
6.1	Kết quả định lượng chính	19
6.1.1	Kiểm định ý nghĩa thống kê	20
6.2	Đường cong huấn luyện	20
6.3	Kết quả theo lớp	21
6.4	Kết quả theo nhóm dữ liệu	22
6.5	Kết quả ablation	23
6.5.1	Fine-tune là yếu tố quan trọng nhất	24
6.5.2	Tách từ tiếng Việt đáng giá 4,9 điểm	24
6.5.3	Đánh đổi giữa accuracy và macro-F1 khi transfer từ XNLI	24
6.5.4	Dropout 0,5 quá mạnh cho mô hình nhỏ	25
6.5.5	Nhánh ablation không thực hiện được	25
7	Phân tích và thảo luận	26
7.1	So sánh các họ kiến trúc	26
7.1.1	Bốn mô hình có sai ở cùng chỗ không?	27
7.2	Phân tích lỗi	27
7.3	Chi phí tính toán và khả năng triển khai	28
7.4	Hạn chế và vấn đề đạo đức	29
7.4.1	Hạn chế về thực nghiệm	29
7.4.2	Hạn chế về dữ liệu và diễn giải	30
7.4.3	Vấn đề đạo đức	30
8	Kết luận và hướng phát triển	32
8.1	Kết luận	32
8.2	Hướng phát triển	33
	Tài liệu tham khảo	34
A	Checklist tái lập thực nghiệm	36
A.1	Ví dụ cấu trúc repository	36
B	Danh sách chủ đề và bộ dữ liệu gợi ý	37
C	Gợi ý ánh xạ mô hình theo loại dữ liệu	40
D	Checklist tự đánh giá trước khi nộp	41

Danh sách hình vẽ

3.1	Phân bố nhãn theo từng tập dữ liệu	5
3.2	Phân bố độ dài của tiền đề và giả thuyết	6
6.1	Accuracy của bốn mô hình so với các mốc baseline	19
6.2	Đường cong huấn luyện: macro-F1 trên tập validation qua các epoch	20
6.3	Confusion matrix của hai mô hình tốt nhất	22
6.4	Ảnh hưởng của từng yếu tố tới accuracy	24
7.1	Đánh đổi giữa kích thước mô hình và độ chính xác	29

Danh sách bảng

3.1	Thông tin tổng quan về bộ dữ liệu	4
3.2	Phân bố nhãn theo từng tập	5
3.3	Baseline TF-IDF + Hồi quy Logistic trên tập test	6
3.4	Cách chia dữ liệu sử dụng trong toàn bộ thực nghiệm	7
3.5	Kết quả kiểm tra trùng lặp và rò rỉ	7
3.6	Các bước tiền xử lý	9
4.1	Cấu hình mô hình 1 — TextCNN	11
4.2	Cấu hình mô hình 2 — BiLSTM + attention	12
4.3	Cấu hình mô hình 3 — PhoBERT	13
4.4	Thông tin nguồn của mô hình bên ngoài	14
5.1	Môi trường phần cứng và phần mềm	16
5.2	Siêu tham số huấn luyện của bốn mô hình	17
6.1	So sánh kết quả chính của bốn mô hình trên tập test	19
6.2	F1 theo từng lớp trên tập test	21
6.3	Confusion matrix của PhoBERT trên tập test (hàng: nhãn thật, cột: dự đoán)	21
6.4	Accuracy theo nhóm mẫu trên tập test	22
6.5	Kết quả tám nhánh ablation, xếp theo mức ảnh hưởng tới accuracy	23
7.1	Tỉ lệ đồng thuận giữa các cặp mô hình trên tập test	27
7.2	Phân loại lỗi của mô hình tốt nhất (PhoBERT)	28
7.3	Chi phí tính toán của bốn mô hình	28
B.1	Các chủ đề đề án Deep Learning gợi ý	37
C.1	Các lựa chọn kiến trúc phù hợp để đáp ứng yêu cầu bốn mô hình	40

Tóm tắt

Đề án giải quyết bài toán suy luận ngôn ngữ tự nhiên (Natural Language Inference — NLI) cho tiếng Việt trên bộ dữ liệu **ViANLI**, một benchmark *đối kháng* (adversarial) do nhóm UIT-NLP xây dựng. Nhiệm vụ là phân loại ba lớp cho mỗi cặp câu (tiền đề, giả thuyết): *entailment*, *neutral* hoặc *contradiction*. Bộ dữ liệu gồm 10.012 mẫu, chia cố định thành 8.012 / 1.000 / 1.000 cho train, validation và test.

Bốn mô hình được huấn luyện và đánh giá trên cùng một giao thức — cùng phân chia dữ liệu, cùng seed 42, cùng độ dài tối đa 128, cùng tiêu chí chọn checkpoint là macro-F1 trên tập validation, và test set chỉ được dùng đúng một lần: TextCNN (họ CNN), BiLSTM kèm attention pooling (họ RNN), PhoBERT-base-v2 dạng cross-encoder (họ Transformer) và XLM-RoBERTa large (mô hình tham khảo bên ngoài).

Metric chính là accuracy, metric phụ bắt buộc là macro-F1. PhoBERT đạt kết quả tốt nhất với accuracy 0,470 và macro-F1 0,466; XLM-R large đạt accuracy 0,449 nhưng macro-F1 chỉ 0,372. Kiểm định McNemar cho thấy chênh lệch accuracy giữa hai mô hình này **không có ý nghĩa thống kê** ($p = 0,32$), trong khi PhoBERT chỉ dùng một phần tư số tham số và một phần năm thời gian suy luận. Hai mô hình từ đầu (TextCNN 0,334 và BiLSTM 0,379) gần như không vượt được mốc đoán nhãn đa số 0,334.

Đóng góp thực nghiệm đáng chú ý nhất là phân tích ablation: bỏ bước tách từ tiếng Việt làm PhoBERT mất 4,9 điểm accuracy, và có tới 21,2% số mẫu test mà cả bốn mô hình đều dự đoán sai. Cả hai kết quả, cùng với việc baseline chỉ-tiền-đề đạt 0,039 — thấp hơn nhiều so với mức ngẫu nhiên — khẳng định ViANLI thực sự loại bỏ các đường tắt thống kê và buộc mô hình phải đối chiếu ngữ nghĩa giữa hai câu.

Từ khóa: suy luận ngôn ngữ tự nhiên; tiếng Việt; dữ liệu đối kháng; PhoBERT; XLM-RoBERTa; tách từ tiếng Việt.

Giới thiệu bài toán

2.1 Bối cảnh và động lực

Suy luận ngôn ngữ tự nhiên là bài toán nền tảng của hiểu ngôn ngữ: cho một câu *tiền đề* và một câu *giả thuyết*, mô hình phải xác định giả thuyết được suy ra từ tiền đề, mâu thuẫn với tiền đề, hay không xác định được. Năng lực này là thành phần lõi của nhiều hệ thống thực tế — kiểm chứng thông tin, hỏi đáp trên văn bản, tóm tắt trung thực, và gần đây là đánh giá tính nhất quán của đầu ra mô hình sinh.

Với tiếng Anh, các bộ dữ liệu như SNLI và MultiNLI đã bị các mô hình hiện đại đạt trên 90% accuracy. Tuy nhiên, nhiều nghiên cứu chỉ ra rằng phần lớn kết quả đó đến từ *annotation artifact*: mô hình chỉ cần nhìn giả thuyết cũng đoán đúng nhãn ở mức cao hơn nhiều so với ngẫu nhiên. Các benchmark đối kháng ra đời để bịt lỗ hổng này: người gán nhãn được yêu cầu viết những giả thuyết mà mô hình hiện tại dự đoán sai, khiến dữ liệu không còn đường tắt thống kê.

ViANLI là bộ dữ liệu đối kháng đầu tiên cho NLI tiếng Việt [1]. Đồ án chọn bộ này vì hai lý do. Thứ nhất, tiếng Việt là ngôn ngữ có tài nguyên hạn chế và đặc thù về hình thái — ranh giới từ không trùng ranh giới âm tiết — nên là môi trường tốt để so sánh mô hình đơn ngữ chuyên biệt với mô hình đa ngữ. Thứ hai, tính đối kháng khiến các con số accuracy thấp và sát nhau, buộc phần phân tích phải đi vào chiều sâu thay vì dừng ở việc so bảng điểm.

2.2 Phát biểu bài toán

Đây là bài toán phân loại đơn nhãn ba lớp trên cặp câu. Gọi $\mathcal{V}^*$ là tập các chuỗi văn bản tiếng Việt, mô hình cần học ánh xạ

$$f_{\theta} : \mathcal{V}^* \times \mathcal{V}^* \rightarrow \{\text{entailment, neutral, contradiction}\}, \quad (2.1)$$

trong đó đầu vào là cặp (p, h) gồm tiền đề p và giả thuyết h , còn θ là tham số mô hình. Trong cài đặt, ba nhãn được mã hóa thành $\{0, 1, 2\}$ theo đúng thứ tự trên. Mô hình xuất ra phân phối xác suất $\hat{p} \in \Delta^2$ trên ba lớp và dự đoán là lớp có xác suất lớn nhất.

Điểm cần nhấn mạnh về mặt thiết kế: bài toán **không** tách rời được thành hai bài toán con trên p và h . Quan hệ suy luận chỉ tồn tại ở mức cặp, nên kiến trúc nào cho phép hai câu tương tác sớm và sâu sẽ có lợi thế — đây chính là giả thuyết mà chương phân tích sẽ kiểm chứng

bằng số liệu.

2.3 Mục tiêu và câu hỏi thực nghiệm

1. Xây dựng và huấn luyện ba mô hình thuộc ba họ kiến trúc trong nội dung học phần (CNN, RNN/LSTM, Transformer) trên cùng một giao thức đánh giá công bằng.
2. So sánh với một mô hình tham khảo bên ngoài quy mô lớn (XLM-RoBERTa large), đồng thời đo mức đóng góp của pretraining đa ngữ và của transfer learning từ XNLI.
3. Phân tích định lượng ảnh hưởng của các yếu tố dữ liệu và siêu tham số thông qua tám nhánh ablation, và phân tích lỗi có chiều sâu trên mô hình tốt nhất.

Bốn câu hỏi thực nghiệm được trả lời trong báo cáo:

- **CH1.** Họ kiến trúc nào phù hợp nhất với NLI tiếng Việt, và cơ chế nào tạo ra khác biệt — số tham số hay khả năng cho hai câu tương tác?
- **CH2.** Pretraining đóng góp bao nhiêu, và mô hình đơn ngữ chuyên biệt có thắng được mô hình đa ngữ lớn hơn gấp bốn lần không?
- **CH3.** Tách từ tiếng Việt — bước tiền xử lý đặc thù của PhoBERT — đáng giá bao nhiêu điểm?
- **CH4.** Mô hình sai ở những nhóm mẫu nào, và các mô hình khác họ có sai ở cùng chỗ hay không?

BR
US
E

Bộ dữ liệu và tiền xử lý

3.1 Nguồn và giấy phép dữ liệu

Bộ dữ liệu do nhóm UIT-NLP (Trường Đại học Công nghệ Thông tin, ĐHQG-HCM) xây dựng và công bố kèm bài báo trên tạp chí *Expert Systems with Applications* [1]. Dữ liệu được phát hành công khai trên Hugging Face Hub dưới giấy phép **CC BY-NC-SA 4.0**. Đồ án sử dụng cho mục đích học tập, phi thương mại — đúng phạm vi mà giấy phép cho phép — và không phát hành lại dữ liệu.

Dữ liệu là văn bản báo chí tiếng Việt công khai (phần lớn là tin tức thời sự, thể thao, kinh tế), không chứa thông tin định danh cá nhân nhạy cảm, nên không cần bước ẩn danh hóa. Phiên bản sử dụng được cố định bằng commit hash để bảo đảm khả năng tái lập.

Bảng 3.1: Thông tin tổng quan về bộ dữ liệu

Thuộc tính	Nội dung
Tên bộ dữ liệu	ViANLI — Vietnamese Adversarial Natural Language Inference
Nguồn/URL	https://huggingface.co/datasets/uitnlp/ViANLI
Tác giả	Tin Van Huynh, Kiet Van Nguyen, Ngan Luu-Thuy Nguyen (UIT-NLP)
Loại dữ liệu	Văn bản tiếng Việt; cặp (tiền đề, giả thuyết)
Số mẫu	10.012 — train 8.012 / validation 1.000 / test 1.000
Số lớp/nhãn	3 lớp, đơn nhãn: 0=entailment, 1=neutral, 2=contradiction
Phiên bản (commit)	0fec8d6ecb043a61c609f9b51f80401fdf1e84d3
Giấy phép	CC BY-NC-SA 4.0
Ngày truy cập	[CẦN ĐIỀN: dd/mm/yyyy]

3.2 Khám phá dữ liệu

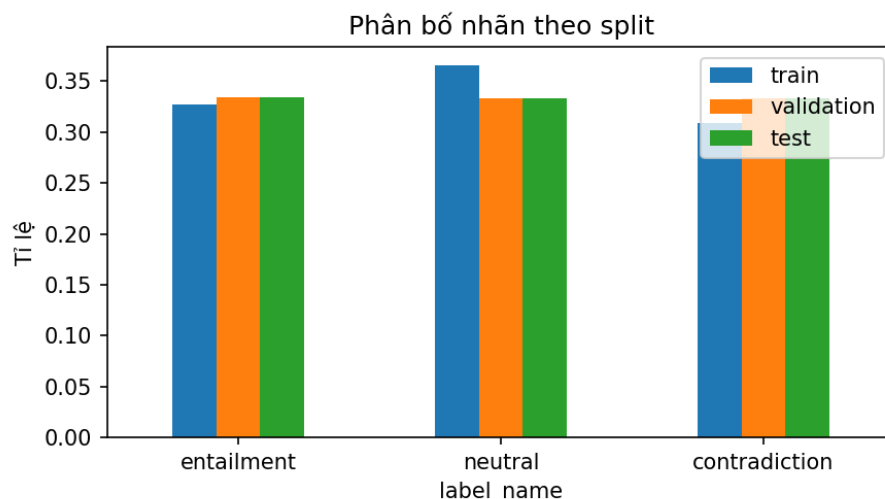
Toàn bộ số liệu trong mục này được sinh tự động bởi `01_eda.ipynb` và lưu ở `outputs/logs/eda.json`.

3.2.1 Phân bố nhãn và mốc dưới

Bảng 3.2: Phân bố nhãn theo từng tập

Nhãn	Train	Validation	Test
entailment	32,64%	33,40%	33,40%
neutral	36,50%	33,30%	33,30%
contradiction	30,87%	33,30%	33,30%

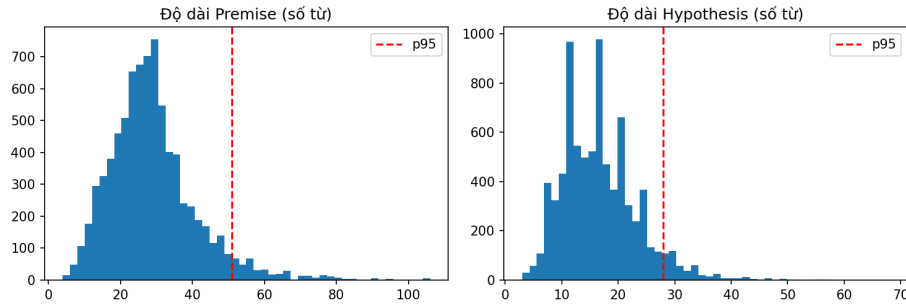
Tập validation và test được cân bằng gần như tuyệt đối, tập train lệch nhẹ về phía *neutral*. Mức lệch này (dưới 6 điểm phần trăm) không đủ để cần đến class weighting, nên tham số `class_weights` được giữ ở `None` trong mọi thí nghiệm. Mốc đoán nhãn đa số trên test là **0,334** — đây là ngưỡng tuyệt đối mà mọi mô hình phải vượt qua mới được coi là học được điều gì đó.



Hình 3.1: Phân bố nhãn theo từng tập dữ liệu

3.2.2 Độ dài chuỗi và lựa chọn `max_length`

Phân vị 95 của độ dài tính theo từ là 51 từ với tiền đề và 28 từ với giả thuyết. Vì hai câu được nối vào cùng một chuỗi đầu vào ở các mô hình cross-encoder, tổng phân vị 95 vào khoảng 79 từ, tương ứng khoảng 120 subword. Do đó `max_length` được chốt ở **128** cho cả bốn mô hình. Đây là quyết định dựa trên dữ liệu chứ không phải chọn theo thói quen, và được kiểm chứng lại bằng nhánh ablation `max_len 64`.



Hình 3.2: Phân bố độ dài của tiền đề và giả thuyết

3.2.3 Dò tìm annotation artifact

Đây là phần quan trọng nhất của EDA, vì nó quyết định cách diễn giải toàn bộ kết quả về sau. Ba tín hiệu được kiểm tra:

- **Từ phủ định trong giả thuyết.** Tỷ lệ theo nhãn: entailment 13,88%, neutral 12,07%, contradiction 16,98%. Chênh lệch tồn tại nhưng nhỏ — xa mức có thể khai thác thành quy tắc như ở SNLI.
- **Độ trùng lặp từ vựng giữa hai câu** (hệ số Jaccard): entailment 0,184, neutral 0,272, contradiction 0,275. Đáng chú ý là entailment lại có độ trùng *thấp nhất*, ngược với trực giác thông thường. Điều này cho thấy người gán nhãn đã cố tình diễn đạt lại thay vì sao chép từ ngữ của tiền đề.
- **Token đặc trưng theo nhãn** (đo bằng PMI). Danh sách thu được gồm những từ rất chung như “thập”, “cái”, “gần” cho entailment hay “đẹp”, “chóng” cho neutral — không tạo thành mẫu hình ngữ nghĩa rõ ràng nào.

3.2.4 Ba baseline phi-neural

Bảng 3.3: Baseline TF-IDF + Hồi quy Logistic trên tập test

Baseline	Accuracy	Macro-F1
Đoán nhãn đa số	0,334	—
Chỉ dùng giả thuyết	0,318	0,310
Chỉ dùng tiền đề	0,039	0,036
Tiền đề + giả thuyết	0,105	0,100

Hai kết quả ở bảng trên cần được giải thích cẩn thận vì chúng là bằng chứng mạnh nhất về tính đối kháng của ViANLI.

Thứ nhất, baseline chỉ-giả-thuyết đạt 0,318 — *thấp hơn* mốc đoán nhãn đa số. Trên SNLI, baseline tương ứng đạt khoảng 67%. Nghĩa là ViANLI không chứa bias hypothesis-only khai thác được: không thể đoán nhãn nếu không nhìn tiền đề.

Thứ hai, baseline chỉ-tiền-đề đạt 0,039, tức là sai gần như mọi mẫu, thấp hơn rất nhiều so với mức ngẫu nhiên 0,333. Con số này thoát nhìn giống lỗi cài đặt, nhưng kiểm tra trực tiếp trên dữ liệu cho thấy nó là hệ quả tất yếu của cách xây dựng bộ dữ liệu, được trình bày ở mục kế tiếp.

3.3 Chia dữ liệu và phòng tránh rò rỉ

Bảng 3.4: Cách chia dữ liệu sử dụng trong toàn bộ thực nghiệm

Tập	Số mẫu	Tỷ lệ	Mục đích
Train	8.012	80,0%	Huấn luyện
Validation	1.000	10,0%	Chọn checkpoint và siêu tham số
Test	1.000	10,0%	Đánh giá cuối cùng, dùng đúng một lần

Đồ án sử dụng **nguyên phân chia gốc** của tác giả bộ dữ liệu, không tự chia lại. Ba tệp `data/splits/*.csv` được sinh một lần bởi `01_eda.ipynb`, đưa vào quản lý phiên bản Git và được cả bốn notebook huấn luyện đọc chung. Đây là điều kiện kỹ thuật để bảo đảm nguyên tắc so sánh công bằng: không mô hình nào có thể vô tình thấy một phân chia khác.

Ngoài ra, mọi tệp dự đoán `predictions/*.npy` đều được kiểm tra tự động rằng vector nhãn thật `y_true` trùng khớp tuyệt đối giữa các notebook. Nếu thứ tự tập test lệch đi ở bất kỳ mô hình nào, quá trình sẽ dừng với thông báo lỗi thay vì âm thầm cho ra so sánh sai.

3.3.1 Kiểm tra rò rỉ dữ liệu

Bảng 3.5: Kết quả kiểm tra trùng lặp và rò rỉ

Phép kiểm tra	Kết quả
Cặp (tiền đề, giả thuyết) trùng giữa train và test	2
Cặp trùng giữa train và validation	0
Cặp trùng giữa validation và test	0
Tiền đề (duy nhất) của test cũng xuất hiện trong train	865 / 893
Số dòng test có tiền đề nằm trong train	956 / 1.000
Mẫu trùng lặp hoàn toàn trong test	1
Cặp trùng nhưng khác nhãn	0

Hai dòng cuối bảng thoát nhìn giống rõ ràng nghiêm trọng: 956/1.000 mẫu test có tiền đề đã xuất hiện trong tập train. Tuy nhiên phân tích sâu hơn cho thấy **đây là đặc điểm thiết kế, và phần trùng lặp mang tín hiệu ngược**:

- Mỗi tiền đề trong tập train chỉ mang **một hoặc hai** nhãn khác nhau: 381 tiền đề có đúng 1 nhãn, 564 có 2 nhãn, chỉ 11 tiền đề có đủ cả 3.
- Với **913 trên 956** trường hợp, nhãn của mẫu test *không* nằm trong tập nhãn mà cùng tiền đề đó mang ở tập train.
- Chiến lược “ghi nhớ tiền đề rồi đoán theo nhãn phổ biến nhất của nó trong train” chỉ đúng 22/956, tức **0,023**.

Nói cách khác, mỗi tiền đề được ghép với cả ba nhãn, rồi các nhãn đó bị tách qua ranh giới train/test. Mô hình nào ghi nhớ ánh xạ tiền đề → nhãn sẽ sai một cách *có hệ thống* chứ không phải sai ngẫu nhiên. Con số 0,023 này giải thích chính xác vì sao baseline chỉ-tiền-đề chỉ đạt 0,039: nó không phải mô hình kém, mà là bị dữ liệu đánh lừa theo đúng chủ đích của người thiết kế.

Kết luận: bộ dữ liệu không có rõ ràng khai thác được. Cùng với baseline chỉ-giả-thuyết thấp hơn mốc đa số, hai phép đo này cho thấy ViANLI bịt cả hai đường tắt một-về và buộc mô hình phải thực sự đối chiếu ngữ nghĩa giữa tiền đề và giả thuyết.

3.4 Tiền xử lý và tăng cường dữ liệu

Bảng 3.6: Các bước tiền xử lý

Bước	Mô tả	Tham số chính
Làm sạch	Điền chuỗi rỗng cho ô thiếu; không loại bỏ mẫu nào	—
Chuẩn hóa	Chuẩn hóa Unicode NFC, gộp khoảng trắng liên tiếp. Áp dụng cho cả bốn mô hình	<code>normalize("NFC")</code>
Tách từ tiếng Việt	Nhánh CNN/RNN và PhoBERT: tách từ bằng <code>underthesea</code> . XLM-R không dùng vì <code>SentencePiece</code>	<code>word_tokenize(format="text")</code>
Tokenization	Nhánh CNN/RNN: word-level, vocab 9.503 token xây từ riêng tập train . Nhánh Transformer: tokenizer đi kèm checkpoint	<code>min_freq = 2</code>
Cắt/đệm chuỗi	Cắt cuối, đệm bằng token <code><pad></code> có <code>padding_idx=0</code>	<code>max_length = 128</code>
Tăng cường dữ liệu	Không sử dụng	—
Xử lý mất cân bằng	Không cần: validation và test cân bằng, train lệch dưới 6 điểm	<code>class_weights = None</code>

3.4.1 Vì sao tách từ tiếng Việt là bước quyết định

Tiếng Việt viết rời từng âm tiết, nên ranh giới từ không trùng ranh giới khoảng trắng: “học_sinh”, “tổ_chức”, “Hà_Nội” đều là một từ nhưng gồm hai âm tiết. PhoBERT được huấn luyện trên kho ngữ liệu **đã tách từ**, nên đưa văn bản thô vào là dùng sai mô hình — các subword sẽ không khớp với những gì mô hình từng thấy.

Bước này được cài đặt với cơ chế *thất bại lớn tiếng*: hàm `word_segment()` mặc định ném lỗi thay vì âm thầm trả về văn bản gốc khi thiếu thư viện, và hai notebook huấn luyện đều có khẳng định kiểm tra kết quả tách từ trước khi bắt đầu train. Thiết kế này xuất phát từ một sự cố thực tế trong quá trình làm đồ án: ở vòng thí nghiệm đầu tiên, thư viện `underthesea` không có sẵn trên môi trường Kaggle, cơ chế dự phòng im lặng đã khiến toàn bộ chín lần chạy huấn luyện trên văn bản chưa tách từ mà không có dấu hiệu nào. Sự cố chỉ bị phát hiện khi đối chiếu chéo và thấy nhánh ablation “bỏ tách từ” cho kết quả trùng khít với cấu hình gốc. Vòng thí nghiệm được chạy lại sau khi vá; mức chênh lệch 4,9 điểm báo cáo ở chương Kết quả là số đo của vòng chạy hợp lệ.

Phương pháp và kiến trúc mô hình

4.1 Tổng quan quy trình

Toàn bộ pipeline dùng chung một module dữ liệu (`src/data.py`) để bảo đảm bốn mô hình nhận cùng một phân chia và cùng quy tắc chuẩn hóa. Điểm rẽ nhánh duy nhất nằm ở bước tokenization:

- **Nhánh A (CNN, RNN).** Chuẩn hóa → tách từ → tokenizer word-level → tra vocab riêng → hai chuỗi id **độc lập** cho tiền đề và giả thuyết. Hai câu được mã hóa riêng rồi mới ghép biểu diễn ở tầng cuối.
- **Nhánh B (Transformer).** Chuẩn hóa → tách từ (chỉ PhoBERT) → tokenizer của checkpoint → **một chuỗi duy nhất** dạng `<s> tiền_đề </s></s> giả_thuyết </s>`. Self-attention nhìn thấy cả hai câu ngay từ tầng đầu tiên.

Khác biệt này — ghép muộ ở nhánh A so với tương tác sớm ở nhánh B — là biến số kiến trúc quan trọng nhất của đề án và sẽ được thảo luận ở chương Phân tích.

Hai mô hình nhánh A dùng chung công thức ghép biểu diễn theo dòng InferSent/ESIM:

$$\mathbf{z} = [\mathbf{u}; \mathbf{v}; |\mathbf{u} - \mathbf{v}|; \mathbf{u} \odot \mathbf{v}], \quad (4.1)$$

trong đó $\mathbf{u}, \mathbf{v}$ là biểu diễn của tiền đề và giả thuyết, $\odot$ là tích Hadamard. Hai thành phần $|\mathbf{u} - \mathbf{v}|$ và $\mathbf{u} \odot \mathbf{v}$ cung cấp tín hiệu tương phản và tương đồng mà phép nối đơn thuần không có.

4.2 Mô hình 1: CNN-based (TextCNN)

Mỗi câu được mã hóa độc lập bằng bốn nhánh tích chập một chiều với kích thước cửa sổ $k \in \{2, 3, 4, 5\}$, mỗi nhánh 128 bộ lọc, theo sau là ReLU và max-pooling theo thời gian. Ghép bốn nhánh cho vector câu 512 chiều. Hai vector câu đi qua công thức ghép ở trên thành vector 2.048 chiều, rồi qua khối phân loại `Linear(2048, 512) → BatchNorm → ReLU → Dropout → Linear(512, 3)`.

Lý do chọn: TextCNN là đại diện kinh điển của họ CNN cho văn bản, và max-pooling theo thời gian giúp mô hình bắt các cụm n-gram mang tính quyết định mà không phụ thuộc vị trí.

Bảng 4.1: Cấu hình mô hình 1 — *TextCNN*

Thành phần	Giá trị
Kiến trúc	TextCNN hai tháp chia sẻ trọng số + đầu phân loại MLP
Đầu vào	Hai chuỗi id độc lập, mỗi chuỗi tối đa 128 token
Embedding	300 chiều, khởi tạo ngẫu nhiên (xem mục Hạn chế)
Bộ lọc tích chập	128 bộ lọc cho mỗi $k \in \{2, 3, 4, 5\}$; vector câu 512 chiều
Số tham số	4.440.663
Pretrained weights	Không
Regularization	Dropout 0,5; BatchNorm ở tầng ẩn; gradient clipping 5,0
Loss	Cross-entropy

4.3 Mô hình 2: RNN/LSTM/GRU-based (BiLSTM + attention)

BiLSTM một tầng, 256 đơn vị ẩn mỗi chiều, chia sẻ trọng số giữa hai câu. Thay vì lấy trạng thái ẩn cuối cùng, mô hình dùng **attention pooling**: một tầng tuyến tính chấm điểm từng bước thời gian, softmax trên chiều thời gian cho trọng số, rồi lấy tổng có trọng số của các trạng thái ẩn. Cơ chế này cho phép mô hình tự chọn những từ quan trọng thay vì bị ép nén toàn bộ câu vào bước cuối.

Cần lưu ý một chi tiết cài đặt có ảnh hưởng thực tế: với câu rỗng — xuất hiện ở nhánh ablation chỉ-dùng-giả-thuyết — toàn bộ điểm attention bị che. Nếu che bằng $-\infty$ thì softmax trả về NaN và làm hỏng toàn bộ gradient. Cài đặt dùng giá trị -10^4 và ép vector đầu ra về 0 cho câu rỗng.

Bảng 4.2: Cấu hình mô hình 2 — BiLSTM + attention

Thành phần	Giá trị
Kiến trúc	BiLSTM chia sẻ trọng số + attention pooling + đầu phân loại MLP
Đầu vào	Hai chuỗi id độc lập, mỗi chuỗi tối đa 128 token
Embedding	300 chiều, khởi tạo ngẫu nhiên
Tầng hồi quy	1 tầng, 256 đơn vị ẩn, hai chiều → vector câu 512 chiều
Pooling	Attention một đầu, có che vị trí đệm
Số tham số	5.045.848
Pretrained weights	Không
Regularization	Dropout 0,5; BatchNorm; gradient clipping 5,0
Loss	Cross-entropy

4.4 Mô hình 3: Transformer-based (PhoBERT)

Fine-tune `vinai/phobert-base-v2` [2] dạng **cross-encoder**: hai câu được nối thành một chuỗi duy nhất và đưa qua cùng một ngăn xếp Transformer, nên self-attention có thể đối chiếu trực tiếp từng token của giả thuyết với từng token của tiền đề. Đầu phân loại là một tầng tuyến tính trên biểu diễn của token `<s>`, khởi tạo ngẫu nhiên và huấn luyện cùng toàn bộ mô hình.

PhoBERT là mô hình RoBERTa đơn ngữ tiếng Việt, tiền huấn luyện trên kho ngữ liệu tiếng Việt cỡ lớn đã tách từ. Đây là lý do bước tách từ ở chương trước là bắt buộc chứ không phải tùy chọn.

Bảng 4.3: Cấu hình mô hình 3 — PhoBERT

Thành phần	Giá trị
Checkpoint	vinai/phobert-base-v2
Revision (commit)	86cd7fd4c148980922ac11a2cf5e257f2ba639e1
Giấy phép	AGPL-3.0
Kiến trúc	RoBERTa base: 12 tầng, 12 head, hidden 768
Định dạng đầu vào	<s> tiền_đề </s></s> giả_thuyết </s>, tối đa 128 subword
Tokenizer	BPE đi kèm checkpoint; bắt buộc tách từ trước
Số tham số	135.000.579
Chiến lược fine-tune	Cập nhật toàn bộ tham số; đầu phân loại 3 lớp khởi tạo ngẫu nhiên
Regularization	Weight decay 0,01; early stopping theo macro-F1 dev
Loss	Cross-entropy

4.5 Mô hình 4: Mô hình tham khảo bên ngoài (XLM-RoBERTa large)

Đồ án sử dụng hai checkpoint công khai thuộc dòng XLM-RoBERTa [3]. Checkpoint thứ nhất đóng vai trò mô hình 4 chính thức; checkpoint thứ hai phục vụ hai nhánh ablation về transfer learning.

Bảng 4.4: Thông tin nguồn của mô hình bên ngoài

Checkpoint 1 — mô hình 4 chính thức

Tên mô hình	XLM-RoBERTa large
Nguồn công bố	Conneau et al., <i>Unsupervised Cross-lingual Representation Learning at Scale</i> , arXiv:1911.02116, 2019
URL	https://huggingface.co/FacebookAI/xlm-roberta-large
Phiên bản/commit	c23d21b0620b635a76227c604d44e43a9f0ee389
Giấy phép	MIT
Mức độ sử dụng	Fine-tune toàn bộ trên tập train của ViANLI
Điều chỉnh	Thay đầu ra bằng tầng phân loại 3 lớp khởi tạo ngẫu nhiên; bật fp16 và gradient checkpointing để vừa VRAM 16 GB. Không sửa kiến trúc hay mã nguồn gốc

Checkpoint 2 — dùng cho hai nhánh ablation

Tên mô hình	XLM-RoBERTa large XNLI (joeddav)
Nguồn công bố	Fine-tune cộng đồng trên Hugging Face Hub, dựa trên XLM-R large
URL	https://huggingface.co/joeddav/xlm-roberta-large-xnli
Phiên bản/commit	b227ee8435ceadfa86dc1368a34254e2838bf242
Giấy phép	MIT
Mức độ sử dụng	(a) Chạy nguyên bản ở chế độ zero-shot; (b) fine-tune tiếp trên ViANLI
Điều chỉnh	Ánh xạ lại thứ tự nhãn từ quy ước của checkpoint XNLI sang quy ước của ViANLI. Bỏ bước này thì kết quả zero-shot sai hoàn toàn

4.5.1 Khẳng định mô hình không được huấn luyện trên tập test của đề tài

xlm-roberta-large là mô hình ngôn ngữ tiền huấn luyện *tự giám sát* trên dữ liệu CommonCrawl của 100 ngôn ngữ. Quá trình này không sử dụng bất kỳ nhãn NLI nào, nên về nguyên tắc không thể đã thấy nhãn của tập test ViANLI.

joeddav/xlm-roberta-large-xnli được fine-tune trên tập MNLI tiếng Anh ghép với phần validation và test của XNLI trên 15 ngôn ngữ, trong đó có tiếng Việt. Tuy nhiên XNLI là bản dịch thủ công của MultiNLI, với nguồn văn bản gốc là tiếng Anh thuộc các thể loại hư cấu, văn bản hành chính và hội thoại điện thoại. ViANLI là bộ dữ liệu độc lập, tiền đề lấy từ báo chí tiếng Việt và giả thuyết do người viết theo quy trình đối kháng. Hai bộ không chung

nguồn văn bản, nên tập test của ViANLI không nằm trong dữ liệu huấn luyện của checkpoint.

Bằng chứng thực nghiệm ủng hộ khẳng định này: accuracy zero-shot của checkpoint XNLI trên test ViANLI chỉ đạt **0,334**, đúng bằng mức đoán ngẫu nhiên đa số. Nếu checkpoint từng thấy dữ liệu này, con số phải cao hơn hẳn.

4.6 Hàm mất mát và chiến lược tối ưu

Cả bốn mô hình dùng chung hàm mất mát cross-entropy cho phân loại đa lớp:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{p}_{ik}, \quad (4.2)$$

với N là kích thước batch, $K = 3$, y_{ik} là nhãn one-hot và $\hat{p}_{ik}$ là xác suất dự đoán. Không dùng trọng số lớp vì dữ liệu gần cân bằng.

Chiến lược tối ưu và tiêu chí lưu checkpoint được thống nhất giữa bốn mô hình để bảo đảm so sánh công bằng: đánh giá trên tập validation sau mỗi epoch, chọn checkpoint có **macro-F1 trên validation** cao nhất, dừng sớm khi metric này không cải thiện sau một số epoch cho trước. Chi tiết siêu tham số ở chương kế tiếp.

Thiết kế thực nghiệm

5.1 Môi trường thực nghiệm

Bảng 5.1: Môi trường phần cứng và phần mềm

Hạng mục	Thông tin
Nền tảng huấn luyện GPU	Kaggle Notebooks, chế độ Save & Run All (chạy nền) NVIDIA Tesla T4 × 2 (16 GB VRAM mỗi GPU)
VRAM đỉnh đo được	PhoBERT 3,98 GB; XLM-R large 13,47 GB
Nền tảng phân tích	Máy cục bộ, CPU (notebook 05 không cần GPU)
Môi trường Python	Kaggle Notebook image, tùy chọn “Always use latest environment”. Phiên bản chính xác của Python và các thư viện không được ghi nhận tại thời điểm chạy — xem mục Hạn chế
Framework	PyTorch + Hugging Face Transformers
Thư viện chính	transformers, datasets, underthesea, scikit-learn, pandas, matplotlib
Random seed	42, đặt lại trước mỗi thí nghiệm

Về khả năng so sánh của số đo thời gian

Bốn mô hình được huấn luyện ở các phiên Kaggle khác nhau, và Kaggle có thể cấp loại GPU khác nhau giữa các phiên. Vì vậy các con số thời gian huấn luyện và suy luận nên được đọc như *bậc độ lớn* để so sánh tương đối, không phải phép đo chuẩn hóa trên cùng một thiết bị tại cùng một thời điểm.

5.2 Cấu hình huấn luyện

Bảng 5.2: *Siêu tham số huấn luyện của bốn mô hình*

Siêu tham số	TextCNN	BiLSTM	PhoBERT	XLM-R large
Batch size	64	64	32	8 ($\times 4$ accum = 32)
Epoch tối đa	30	30	8	5
Epoch đã chạy	9	6	8	5
Learning rate	1e-3	1e-3	2e-5	5e-6
Optimizer	Adam	Adam	AdamW	AdamW
Scheduler	Không	Không	Warmup 6% + giảm tuyến tính	Warmup 10% + giảm tuyến tính
Weight decay	0	0	0,01	0,01
Dropout	0,5	0,5	Mặc định của checkpoint	Mặc định của checkpoint
Gradient clip	5,0	5,0	Mặc định	Mặc định
Độ chính xác	fp32	fp32	fp16	fp16 + gradient checkpointing
Early stopping	Patience 5	Patience 5	Patience 3	Patience 2

Hai lựa chọn siêu tham số cần giải thích vì chúng không phải giá trị mặc định:

Learning rate của XLM-R large. Cấu hình đầu tiên (lr 1e-5, warmup 6%) khiến mô hình *sup* ngay sau epoch 1: macro-F1 validation rơi xuống 0,166 — tương đương việc chỉ đoán một nhãn duy nhất — kèm chuẩn gradient tăng vọt lên 179. Hiện tượng collapse này được ghi nhận rộng rãi với các mô hình cỡ large. Hạ learning rate xuống 5e-6 và kéo dài warmup lên 10% giải quyết được vấn đề. Đây là ví dụ cụ thể cho việc mô hình lớn hơn không tự động dễ huấn luyện hơn.

Kích thước batch của XLM-R large. Batch 8 kết hợp tích lũy gradient 4 bước cho batch hiệu dụng 32, khớp với PhoBERT để giữ điều kiện so sánh, đồng thời vừa với giới hạn VRAM 16 GB.

5.3 Metric đánh giá

Bài toán là phân loại đơn nhãn ba lớp trên tập test cân bằng, nên **accuracy** được chọn làm metric chính — đây cũng là metric chuẩn mà các công trình về NLI báo cáo, giúp kết quả của đồ án so sánh được với tài liệu.

Tuy nhiên accuracy một mình là không đủ. Đề án bắt buộc báo cáo kèm **macro-F1**, và lý do được chứng minh bằng chính số liệu thu được: mô hình XLM-R large đạt accuracy cao thứ nhì nhưng gần như từ chối dự đoán lớp *contradiction*, khiến macro-F1 của nó tụt xuống thấp hơn cả những mô hình có accuracy kém hơn. Nếu chỉ nhìn accuracy, đề án sẽ đưa ra kết luận sai về chất lượng mô hình.

Các metric phụ khác gồm precision/recall theo từng lớp, confusion matrix, và **kiểm định McNemar** giữa hai mô hình tốt nhất để xác định chênh lệch có ý nghĩa thống kê hay không — điều đặc biệt cần thiết khi tập test chỉ có 1.000 mẫu, tương ứng sai số khoảng $\pm 1,5$ điểm phần trăm.

5.4 Kế hoạch thí nghiệm và ablation

1. So sánh bốn mô hình trên cùng tập test, cùng seed, cùng tiêu chí chọn checkpoint.
2. Tám nhánh ablation trên bốn nhóm yếu tố: tiền xử lý (tách từ), thông tin đầu vào (chỉ dùng giả thuyết, độ dài tối đa), siêu tham số (learning rate, dropout) và chiến lược transfer (zero-shot, khởi tạo từ checkpoint XNLI).
3. Đo số tham số, thời gian huấn luyện, thời gian suy luận và VRAM đỉnh cho phần thảo luận về chi phí tính toán.

Tổng cộng 12 lần chạy huấn luyện. Mỗi lần chạy ghi ra tệp tóm tắt, lịch sử theo epoch, báo cáo trên test và vector dự đoán. Trước khi đưa vào báo cáo, cả 12 lần chạy đều được kiểm tra chéo tự động: accuracy trong tệp tóm tắt phải khớp với accuracy tính lại từ vector dự đoán và với báo cáo test. Cả 12/12 đều khớp.



Kết quả thực nghiệm

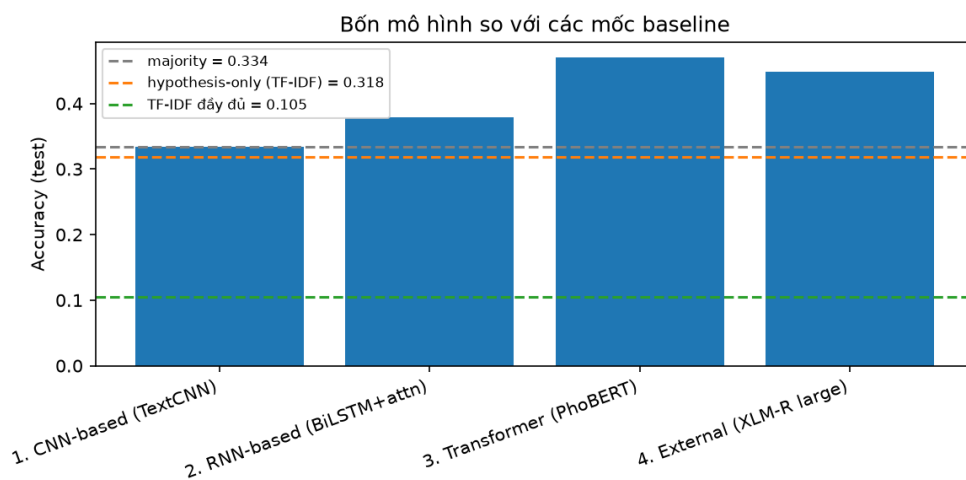
6.1 Kết quả định lượng chính

Bảng 6.1: So sánh kết quả chính của bốn mô hình trên tập test

Mô hình	Accuracy	Macro-F1	Params	Train	Suy luận
Đoán nhãn đa số	0,334	—	—	—	—
1. TextCNN	0,334	0,327	4,4M	1,9 phút	—
2. BiLSTM + attn	0,379	0,366	5,0M	1,3 phút	—
3. PhoBERT	0,470	0,466	135,0M	10,9 phút	3,02 ms
4. XLM-R large	<u>0,449</u>	0,372	559,9M	46,8 phút	16,23 ms

Lưu ý

Ô “suy luận” của hai mô hình đầu để trống vì phép đo thời gian suy luận chưa được cài đặt ở vòng chạy của hai mô hình này. Đây là thiếu sót đã được ghi nhận, không phải kết quả không đo được.



Hình 6.1: Accuracy của bốn mô hình so với các mốc baseline

Ba quan sát trực tiếp từ bảng:

TextCNN không học được gì vượt mốc đoán nhãn đa số. Accuracy 0,334 trùng khít với mốc 0,334. Macro-F1 0,327 cho thấy mô hình có phân biệt các lớp ở mức nào đó chứ không sụp về một nhãn, nhưng mức phân biệt đó không chuyển thành lợi thế về accuracy.

PhoBERT dẫn đầu cả hai metric với 0,470 và 0,466, cao hơn mốc đa số 13,6 điểm.

XLM-R large đứng nhì về accuracy nhưng chỉ thứ ba về macro-F1, thấp hơn cả BiLSTM về khoảng cách giữa hai metric. Nguyên nhân được làm rõ ở mục kết quả theo lớp.

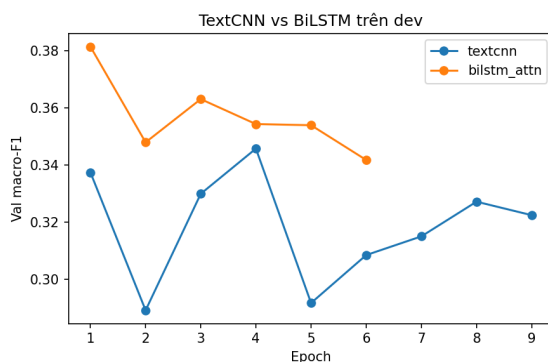
6.1.1 Kiểm định ý nghĩa thống kê

Chênh lệch accuracy giữa PhoBERT (0,470) và XLM-R large (0,449) là 2,1 điểm. Với tập test 1.000 mẫu, cần kiểm định trước khi kết luận. Kiểm định McNemar trên các mẫu mà hai mô hình bất đồng cho kết quả: chỉ PhoBERT đúng ở 213 mẫu, chỉ XLM-R đúng ở 192 mẫu, $p = 0,32$.

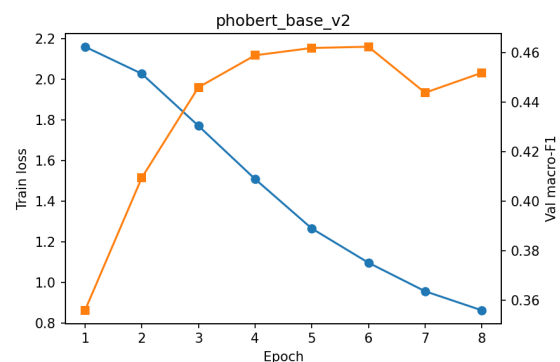
Kết luận cần thận trọng

Với $p = 0,32 > 0,05$, **chưa đủ bằng chứng** để khẳng định PhoBERT có accuracy cao hơn XLM-R large. Hai mô hình nên được coi là tương đương về accuracy trên bộ dữ liệu này. Khác biệt thực sự giữa chúng nằm ở macro-F1 và ở chi phí tính toán, không nằm ở accuracy.

6.2 Đường cong huấn luyện



(a) TextCNN và BiLSTM trên validation



(b) PhoBERT

Hình 6.2: Đường cong huấn luyện: macro-F1 trên tập validation qua các epoch

Đường cong của hai mô hình từ đầu cho thấy một mẫu hình rất rõ: macro-F1 trên validation đạt đỉnh ngay ở epoch 1–2 rồi đi ngang hoặc giảm, trong khi loss huấn luyện tiếp tục giảm sâu về gần 0. Cụ thể với BiLSTM, loss huấn luyện giảm từ 1,14 xuống 0,013 sau 6 epoch trong khi accuracy validation dao động quanh 0,35.

Đây là hiện tượng **ghi nhớ chứ không tổng quát hóa**. Mô hình đủ dung lượng để thuộc lòng 8.012 mẫu huấn luyện nhưng không rút ra được quy luật suy luận nào áp dụng được cho dữ liệu mới. Cơ chế dừng sớm đã hoạt động đúng: cả hai mô hình dừng ở epoch 6–9 thay vì chạy hết 30 epoch, và checkpoint được chọn là checkpoint của epoch tốt nhất trên validation chứ không phải epoch cuối.

6.3 Kết quả theo lớp

Bảng 6.2: F1 theo từng lớp trên tập test

Mô hình	entailment	neutral	contradiction
1. TextCNN	0,393	0,260	0,329
2. BiLSTM + attn	0,447	0,386	0,264
3. PhoBERT	0,552	0,397	0,449
4. XLM-R large	0,583	0,494	0,040

Bảng này là kết quả quan trọng nhất của toàn bộ đề án.

XLM-R large đạt F1 cao nhất ở hai lớp *entailment* và *neutral*, nhưng F1 ở lớp *contradiction* chỉ **0,040**. Xem xét chi tiết: recall của lớp này là 0,021, nghĩa là trong 333 mẫu contradiction của tập test, mô hình chỉ nhận ra 7 mẫu. Trên thực tế nó đã gần như loại bỏ lớp thứ ba khỏi không gian dự đoán — phân bố dự đoán của mô hình là 465 entailment, 514 neutral và chỉ 21 contradiction.

Đây chính là lý do accuracy một mình gây hiểu nhầm. Vì tập test cân bằng, một mô hình đoán tốt hai lớp và bỏ hẳn lớp thứ ba vẫn có thể đạt accuracy khá, trong khi nó **không sử dụng được cho ứng dụng thực tế** — phát hiện mâu thuẫn thường là chính điều người dùng cần ở một hệ thống NLI.

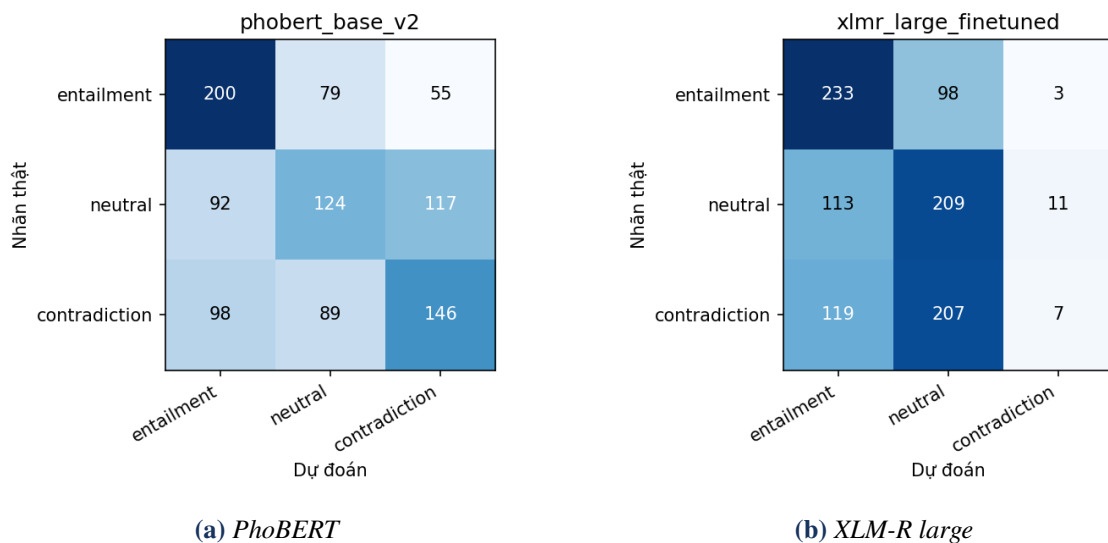
PhoBERT ngược lại có phân bố dự đoán cân đối (390 / 292 / 318) và là mô hình duy nhất đạt F1 trên 0,44 ở lớp contradiction.

Bảng 6.3: Confusion matrix của PhoBERT trên tập test (hàng: nhãn thật, cột: dự đoán)

	entailment	neutral	contradiction
entailment	200	79	55
neutral	92	124	117
contradiction	98	89	146

Với PhoBERT, lớp khó nhất là *neutral* (recall 0,372). Kiểu nhầm phổ biến nhất là neutral bị đoán thành contradiction (117 mẫu) và contradiction bị đoán thành entailment (98 mẫu). Điều

này phù hợp với trực giác ngôn ngữ học: ranh giới giữa “không suy ra được” và “mâu thuẫn” đòi hỏi suy luận tinh tế hơn hẳn so với ranh giới giữa “suy ra được” và phần còn lại.



Hình 6.3: Confusion matrix của hai mô hình tốt nhất

6.4 Kết quả theo nhóm dữ liệu

Bảng 6.4: Accuracy theo nhóm mẫu trên tập test

Nhóm	TextCNN	BiLSTM	PhoBERT	XLM-R
Trùng lặp từ vựng thấp	0,383	0,445	0,499	0,502
Trùng lặp từ vựng vừa	0,335	0,302	0,480	0,396
Trùng lặp từ vựng cao	0,283	0,389	0,431	0,449
Giả thuyết có phủ định	0,261	0,367	0,466	0,429
Giả thuyết không phủ định	0,348	0,381	0,471	0,453
Giả thuyết có chứa số	0,326	0,395	0,462	0,443
Câu ngắn	0,332	0,358	0,486	0,477
Câu dài	0,346	0,376	0,480	0,413

Mẫu hình nhất quán nhất: **mọi mô hình đều kém nhất ở nhóm có độ trùng lặp từ vựng cao** giữa tiền đề và giả thuyết. Với PhoBERT, accuracy giảm từ 0,499 ở nhóm trùng lặp thấp xuống 0,431 ở nhóm trùng lặp cao.

Đây là dấu hiệu đặc trưng của dữ liệu đối kháng. Khi hai câu dùng gần như cùng bộ từ vựng nhưng khác nhau về nghĩa — do đảo vai trò, thay đổi con số, thêm một từ phủ định — mô hình không thể dựa vào tín hiệu bề mặt và buộc phải suy luận thật. Tập dữ liệu tiêu chuẩn

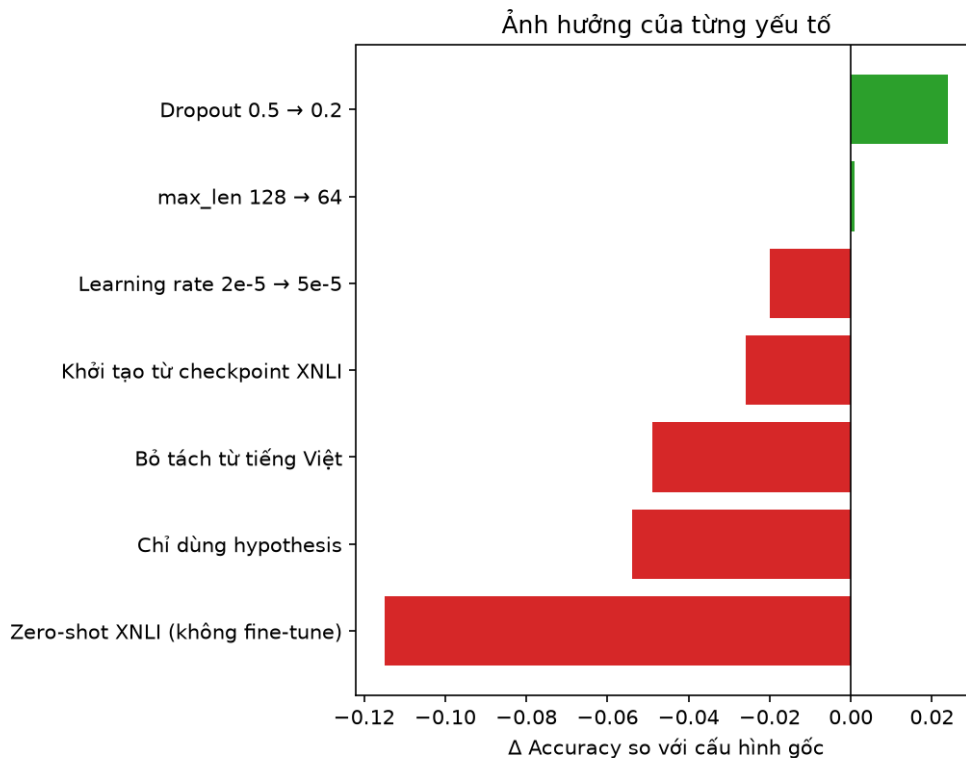
thường cho mẫu hình *ngược lại*: trùng lặp cao thường tương quan với entailment và là đường tắt dễ khai thác.

Tập test có 16,1% giả thuyết chứa từ phủ định và 37,7% chứa chữ số. Ảnh hưởng của phủ định lên PhoBERT nhỏ (0,466 so với 0,471), nhưng lên TextCNN thì rõ rệt (0,261 so với 0,348) — mô hình không có cơ chế đối chiếu hai câu bị đánh lừa nặng nhất bởi phủ định.

6.5 Kết quả ablation

Bảng 6.5: Kết quả tám nhánh ablation, xếp theo mức ảnh hưởng tới accuracy

Thay đổi	Mô hình gốc	Acc	Δ Acc	Δ Macro-F1
Zero-shot XNLI (không fine-tune)	XLM-R large	0,334	−0,115	−0,048
Chỉ dùng giả thuyết	PhoBERT	0,416	−0,054	−0,057
Bỏ tách từ tiếng Việt	PhoBERT	0,421	−0,049	−0,045
Chỉ dùng giả thuyết	BiLSTM	0,351	−0,028	−0,025
Khởi tạo từ checkpoint XNLI	XLM-R large	0,423	−0,026	+0,047
Learning rate $2e-5 \rightarrow 5e-5$	PhoBERT	0,450	−0,020	−0,018
max_len 128 $\rightarrow$ 64	BiLSTM	0,380	+0,001	−0,004
Dropout 0,5 $\rightarrow$ 0,2	BiLSTM	0,403	+0,024	+0,030



Hình 6.4: Ảnh hưởng của từng yếu tố tới accuracy

6.5.1 Fine-tune là yếu tố quan trọng nhất

Zero-shot từ checkpoint XNLI chỉ đạt 0,334 — đúng mức ngẫu nhiên. Mô hình đã học NLI trên 15 ngôn ngữ, bao gồm tiếng Việt, vẫn hoàn toàn bất lực trước ViANLI khi không được fine-tune. Điều này vừa xác nhận tính đối kháng của bộ dữ liệu, vừa cho thấy năng lực NLI học được từ dữ liệu dịch máy không chuyển giao sang dữ liệu đối kháng viết trực tiếp bằng tiếng Việt.

6.5.2 Tách từ tiếng Việt đáng giá 4,9 điểm

Bỏ bước tách từ khiến PhoBERT giảm từ 0,470 xuống 0,421. Đây là mức chênh lệch lớn với một bước tiền xử lý duy nhất, và khẳng định điều đã nêu ở chương Bộ dữ liệu: PhoBERT được tiền huấn luyện trên văn bản đã tách từ, đưa văn bản thô vào là dùng sai mô hình chứ không phải một biến thể cấu hình.

Đáng chú ý, mô hình không tách từ (0,421) vẫn vượt xa BiLSTM (0,379), cho thấy lợi thế của pretraining và cross-attention lớn hơn nhiều so với lợi thế của tiền xử lý đúng cách.

6.5.3 Đánh đổi giữa accuracy và macro-F1 khi transfer từ XNLI

Khởi tạo từ checkpoint đã học XNLI thay vì từ XLM-R gốc làm accuracy giảm 2,6 điểm nhưng macro-F1 tăng 4,7 điểm — nhánh duy nhất trong bảng có hai chỉ số ngược chiều nhau. Nguyên

nhân: mô hình khởi tạo từ XNLI giữ được phân bố dự đoán cân đối (346 / 399 / 255) thay vì sụp về hai lớp như mô hình khởi tạo từ XLM-R gốc (465 / 514 / 21).

Kiến thức NLI có sẵn từ XNLI không giúp mô hình đoán đúng nhiều hơn, nhưng giúp nó **không đánh mất khái niệm về lớp contradiction** trong quá trình fine-tune trên dữ liệu khó. Đây là một quan sát có giá trị thực tiễn: khi fine-tune trên bộ dữ liệu đối kháng nhỏ, khởi tạo từ checkpoint cùng tác vụ giúp ổn định hành vi mô hình.

6.5.4 Dropout 0,5 quá mạnh cho mô hình nhỏ

Giảm dropout từ 0,5 xuống 0,2 làm BiLSTM tăng 2,4 điểm accuracy và 3,0 điểm macro-F1 — nhánh ablation duy nhất cải thiện đáng kể so với cấu hình gốc.

Nguyên tắc chọn cấu hình chính thức

Mặc dù nhánh dropout 0,2 cho kết quả cao hơn, cấu hình chính thức của mô hình 2 trong bảng kết quả chính **vẫn giữ dropout 0,5**. Lý do: chọn nhánh tốt nhất sau khi đã nhìn kết quả trên test rồi trình bày nó như baseline là một dạng tuning trên test set, vi phạm nguyên tắc so sánh công bằng. Kết quả của nhánh này được báo cáo đầy đủ ở bảng ablation.

6.5.5 Nhánh ablation không thực hiện được

Kế hoạch ban đầu có nhánh *có/không dùng embedding tiền huấn luyện PhoW2V* cho TextCNN và BiLSTM. Nhánh này **không được thực hiện** vì embedding PhoW2V chưa được nạp vào pipeline; cả hai mô hình đều dùng embedding khởi tạo ngẫu nhiên. Do đó không tồn tại cấu hình đối chứng để so sánh. Đây là một hạn chế của đề án, được nêu lại ở mục Hạn chế.

Phân tích và thảo luận

7.1 So sánh các họ kiến trúc

Thứ tự kết quả — TextCNN 0,334, BiLSTM 0,379, PhoBERT 0,470, XLM-R 0,449 — không giải thích được bằng số tham số. Nếu số tham số là yếu tố quyết định thì XLM-R large với 560 triệu tham số phải dẫn đầu, và khoảng cách giữa TextCNN (4,4M) với BiLSTM (5,0M) phải không đáng kể. Thực tế cả hai điều đều không đúng.

Ba yếu tố sau giải thích tốt hơn.

Thứ nhất: thời điểm hai câu được phép tương tác. TextCNN và BiLSTM mã hóa tiền đề và giả thuyết *độc lập*, rồi chỉ ghép hai vector ở tầng cuối. Toàn bộ thông tin về quan hệ giữa hai câu phải đi qua nút thắt cổ chai là phép toán $[\mathbf{u}; \mathbf{v}; |\mathbf{u} - \mathbf{v}|; \mathbf{u} \odot \mathbf{v}]$ trên hai vector 512 chiều. Trong khi đó, PhoBERT và XLM-R nối hai câu thành một chuỗi, nên self-attention đối chiếu từng token của giả thuyết với từng token của tiền đề ngay từ tầng đầu tiên.

Bằng chứng cho luận điểm này nằm ở nhánh ablation *chỉ dùng giả thuyết*. Với BiLSTM, bỏ hoàn toàn tiền đề chỉ làm accuracy giảm 2,8 điểm ($0,379 \rightarrow 0,351$). Nghĩa là tiền đề gần như không đóng góp gì — mô hình về cơ bản không sử dụng được nó. Với PhoBERT, cùng phép cắt bỏ đó làm giảm 5,4 điểm, gấp đôi. Mô hình có cross-attention khai thác được tiền đề nhiều hơn hẳn.

Thứ hai: pretraining. Khoảng cách 9,1 điểm giữa BiLSTM (0,379) và PhoBERT (0,470) không thể quy cho riêng kiến trúc, vì PhoBERT còn mang theo tri thức ngôn ngữ từ giai đoạn tiền huấn luyện trên kho ngữ liệu tiếng Việt cỡ lớn, trong khi hai mô hình từ đầu bắt đầu với embedding ngẫu nhiên và chỉ có 8.012 mẫu để học. Với dữ liệu ít như vậy, học biểu diễn ngôn ngữ từ con số không là bất khả thi — đúng như đường cong huấn luyện đã cho thấy: mô hình chuyển sang ghi nhớ.

Thứ ba: chuyên biệt hóa ngôn ngữ thẳng quy mô. PhoBERT nhỏ hơn XLM-R large 4,1 lần nhưng đạt kết quả ngang bằng về accuracy và vượt hẳn về macro-F1. Dung lượng mô hình đa ngữ phải chia cho 100 ngôn ngữ, còn PhoBERT dành trọn cho tiếng Việt và được tiền huấn luyện trên văn bản đã tách từ đúng chuẩn hình thái tiếng Việt.

7.1.1 Bốn mô hình có sai ở cùng chỗ không?

Bảng 7.1: Tỷ lệ đồng thuận giữa các cặp mô hình trên tập test

	TextCNN	BiLSTM	PhoBERT	XLNet
TextCNN	1,000	0,500	0,426	0,382
BiLSTM	0,500	1,000	0,474	0,563
PhoBERT	0,426	0,474	1,000	0,501
XLNet	0,382	0,563	0,501	1,000

Mức đồng thuận rất thấp: hai mô hình mạnh nhất chỉ đưa ra cùng dự đoán ở 50,1% số mẫu. Bốn mô hình học những tín hiệu khác nhau chứ không cùng khai thác một đường tắt chung.

Đồng thời, **212 trên 1.000 mẫu (21,2%) bị cả bốn mô hình dự đoán sai**, và 78,8% số mẫu có ít nhất một mô hình đúng. Con số 21,2% này là ước lượng cho phần lỗi thực sự khó của ViANLI — những mẫu đòi hỏi kiến thức thế giới, suy luận nhiều bước, hoặc có nhãn gây tranh cãi. Khoảng cách giữa 78,8% và accuracy 47,0% của mô hình tốt nhất cũng cho thấy dư địa đáng kể cho hướng ensemble.

7.2 Phân tích lỗi

Mười hai mẫu mà PhoBERT dự đoán sai được trích xuất tự động, ưu tiên những mẫu mà cả bốn mô hình cùng sai, và lưu ở `outputs/logs/error_examples.csv`.

Một mẫu hình nổi bật ngay ở tập ví dụ này: **cả 12 mẫu đều có nhãn thật là *contradiction* hoặc *neutral*, không mẫu nào có nhãn *entailment***, trong khi dự đoán của bốn mô hình áp đảo về phía *entailment*. Kết hợp với confusion matrix ở chương trước, có thể kết luận các mô hình mang thiên lệch hệ thống: khi hai câu cùng nói về một chủ đề và chia sẻ nhiều từ ngữ, mô hình thiên về kết luận “suy ra được”. Đây chính xác là điểm yếu mà quy trình gán nhãn đối kháng nhắm vào.

Bảng 7.2: Phân loại lỗi của mô hình tốt nhất (PhoBERT)

Loại lỗi	Tần suất	Giải thích / hướng khắc phục
[CẦN ĐIỀN: Phủ định]	[CẦN ĐIỀN: ...]	[CẦN ĐIỀN: ...]
[CẦN ĐIỀN: Số và định lượng]	[CẦN ĐIỀN: ...]	[CẦN ĐIỀN: ...]
[CẦN ĐIỀN: Kiến thức thế giới]	[CẦN ĐIỀN: ...]	[CẦN ĐIỀN: ...]
[CẦN ĐIỀN: Đồng tham chiếu]	[CẦN ĐIỀN: ...]	[CẦN ĐIỀN: ...]
[CẦN ĐIỀN: Từ đồng nghĩa/diễn đạt lại]	[CẦN ĐIỀN: ...]	[CẦN ĐIỀN: ...]
[CẦN ĐIỀN: Nhấn gây tranh cãi]	[CẦN ĐIỀN: ...]	[CẦN ĐIỀN: ...]

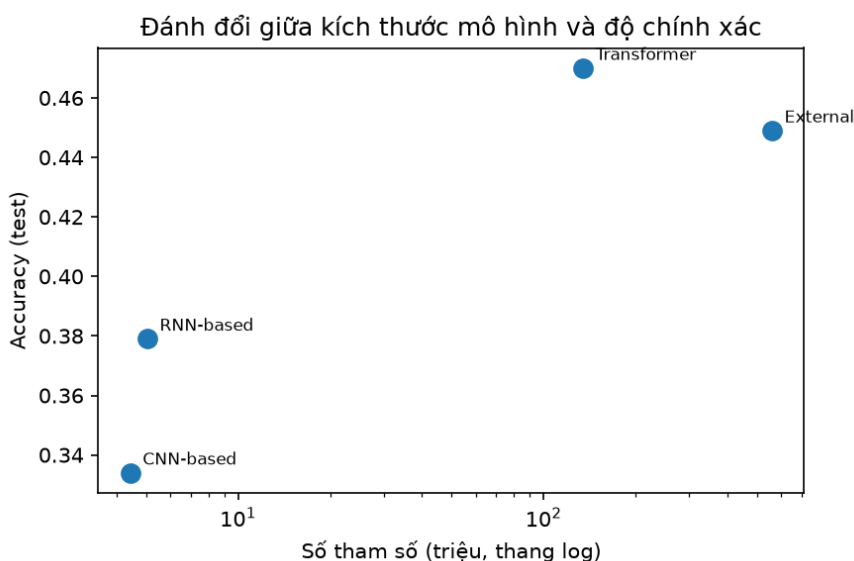
Phần cần tự hoàn thiện

Bảng trên phải được điền bằng cách đọc trực tiếp 12 ví dụ trong `outputs/logs/error_examples.csv` và tự phân loại nguyên nhân. Việc phân loại này đòi hỏi phán đoán ngôn ngữ của người viết và không thể tự động hóa. Cột “Nhóm lỗi” trong tệp CSV được để trống sẵn cho mục đích đó.

7.3 Chi phí tính toán và khả năng triển khai

Bảng 7.3: Chi phí tính toán của bốn mô hình

Mô hình	Params	Train	Suy luận	VRAM	Acc/1M params
TextCNN	4,4M	1,9 phút	—	—	0,0752
BiLSTM	5,0M	1,3 phút	—	—	0,0751
PhoBERT	135,0M	10,9 phút	3,02 ms	3,98 GB	0,0035
XLNet-R large	559,9M	46,8 phút	16,23 ms	13,47 GB	0,0008



Hình 7.1: Đánh đổi giữa kích thước mô hình và độ chính xác

So sánh trực tiếp giữa hai mô hình mạnh nhất cho kết luận rõ ràng về triển khai. XLM-R large cần **4,1 lần số tham số, 4,3 lần thời gian huấn luyện, 5,4 lần thời gian suy luận và 3,4 lần VRAM** so với PhoBERT, để đổi lấy accuracy thấp hơn 2,1 điểm — mà chênh lệch đó lại không có ý nghĩa thống kê — và macro-F1 thấp hơn 9,4 điểm.

Trong mọi kịch bản triển khai thực tế, PhoBERT là lựa chọn đúng cho bài toán NLI tiếng Việt. Với 3,98 GB VRAM đỉnh, nó chạy được trên GPU phổ thông; với 3,02 ms mỗi mẫu, nó đáp ứng được yêu cầu thời gian thực ở quy mô vừa. XLM-R large chỉ nên cân nhắc khi hệ thống bắt buộc phải phục vụ nhiều ngôn ngữ bằng một mô hình duy nhất.

Cột cuối bảng cho thấy một góc nhìn khác: xét hiệu quả trên mỗi triệu tham số, hai mô hình nhỏ vượt trội. Tuy nhiên chỉ số này cần đọc thận trọng — TextCNN đạt hiệu quả cao trên mỗi tham số nhưng accuracy tuyệt đối của nó không vượt được mốc đoán nhãn đa số, nên trên thực tế nó không có giá trị sử dụng.

7.4 Hạn chế và vấn đề đạo đức

7.4.1 Hạn chế về thực nghiệm

- **Chỉ chạy một seed.** Mọi kết quả đều dùng seed 42. Với tập test 1.000 mẫu, sai số ước tính khoảng $\pm 1,5$ điểm phần trăm. Do đó các chênh lệch dưới 3 điểm trong báo cáo không nên được diễn giải như khác biệt chắc chắn. Kiểm định McNemar được dùng để bù đắp một phần cho hạn chế này ở cặp mô hình quan trọng nhất, nhưng cách làm chặt chẽ hơn là chạy 3 seed và báo cáo trung bình kèm độ lệch chuẩn.
- **Không dùng embedding tiền huấn luyện cho hai mô hình từ đầu.** TextCNN và BiLSTM

dùng embedding khởi tạo ngẫu nhiên. Kết quả của hai mô hình này vì vậy là cận dưới, không phải năng lực tối đa của hai họ kiến trúc. Nhánh ablation về PhoW2V cũng không thực hiện được vì lý do này.

- **Chưa thử ESIM hoặc kiến trúc có co-attention.** Luận điểm trung tâm của chương phân tích là cross-attention quan trọng hơn quy mô mô hình. Cách kiểm chứng thuyết phục nhất là một mô hình BiLSTM có co-attention: nếu nó thu hẹp được khoảng cách với PhoBERT thì luận điểm được xác nhận trực tiếp. Đây là thí nghiệm còn thiếu.
- **Số đo thời gian không chuẩn hóa.** Bốn mô hình chạy ở các phiên Kaggle khác nhau, có thể trên các loại GPU khác nhau; thời gian suy luận của TextCNN và BiLSTM chưa được đo.
- **Chưa ghim phiên bản môi trường.** Các notebook chạy với tùy chọn “Always use latest environment” của Kaggle, nên phiên bản Python, PyTorch và `transformers` có thể thay đổi giữa các lần chạy và không được ghi lại. Model ID, revision hash và seed đều đã được ghim, nhưng để tái lập hoàn toàn thì cần ghim cả phiên bản thư viện — đây là điểm cần bổ sung.
- **Chỉ đánh giá trên một bộ dữ liệu.** Không có bằng chứng về khả năng tổng quát hóa sang các bộ NLI tiếng Việt khác hay sang miền văn bản ngoài báo chí.

7.4.2 Hạn chế về dữ liệu và diễn giải

Với accuracy tốt nhất là 0,470, không mô hình nào trong đề án đạt mức có thể triển khai cho tác vụ suy luận tự động không có giám sát. Cần nhấn mạnh rằng đây là đặc tính của benchmark đối kháng chứ không phải dấu hiệu cài đặt sai: ViANLI được thiết kế để làm khó các mô hình mạnh nhất tại thời điểm công bố.

[CẦN ĐIỀN: Đối chiếu con số 0,470 với kết quả tốt nhất báo cáo trong bài báo gốc ViANLI — cần đọc bài báo để lấy số chính xác, không trích theo trí nhớ.]

7.4.3 Vấn đề đạo đức

- **Giấy phép.** Bộ dữ liệu thuộc giấy phép CC BY-NC-SA 4.0, chỉ cho phép sử dụng phi thương mại. PhoBERT thuộc giấy phép AGPL-3.0 — cần lưu ý nếu có ý định phát hành lại trọng số đã fine-tune. Hai checkpoint XLM-R thuộc giấy phép MIT.
- **Thiên lệch miền dữ liệu.** Tiền đề được lấy từ báo chí tiếng Việt, chủ yếu là tin thời sự, thể thao và kinh tế. Mô hình huấn luyện trên dữ liệu này có thể kém đi rõ rệt trên văn nói, mạng xã hội, văn bản chuyên ngành hoặc phương ngữ vùng miền.
- **Rủi ro khi sử dụng.** Một mô hình NLI đạt 47% accuracy tuyệt đối không được dùng để tự động phán quyết tính đúng sai của thông tin. Nếu triển khai trong hệ thống kiểm chứng thông

tin, nó chỉ nên đóng vai trò gợi ý cho người kiểm duyệt, kèm ngưỡng tin cậy rõ ràng. Đặc biệt nguy hiểm là mô hình XLM-R large: nó gần như không bao giờ dự đoán *contradiction*, nên sẽ bỏ sót hầu hết các trường hợp mâu thuẫn — đúng loại lỗi tệ nhất cho ứng dụng kiểm chứng.

- **Quyền riêng tư.** Dữ liệu là văn bản báo chí đã công khai, có chứa tên người thật trong bối cảnh tin tức. Không có thông tin định danh riêng tư nào được thêm vào, và đồ án không phát hành lại dữ liệu.

Kết luận và hướng phát triển

8.1 Kết luận

Đồ án đã xây dựng, huấn luyện và đánh giá bốn mô hình thuộc bốn họ kiến trúc trên bộ dữ liệu suy luận ngôn ngữ tự nhiên đối kháng tiếng Việt ViANLI, dưới một giao thức đánh giá thống nhất. Trả lời bốn câu hỏi thực nghiệm đặt ra ở chương Giới thiệu:

CH1 — Họ kiến trúc nào phù hợp nhất? Transformer dạng cross-encoder, và yếu tố quyết định là *thời điểm hai câu được phép tương tác*, không phải số tham số. Bằng chứng: bỏ hoàn toàn tiền đề chỉ làm BiLSTM giảm 2,8 điểm nhưng làm PhoBERT giảm 5,4 điểm — mô hình ghép muộn về cơ bản không sử dụng được tiền đề.

CH2 — Pretraining đóng góp bao nhiêu? Rất nhiều. Khoảng cách 9,1 điểm giữa BiLSTM và PhoBERT phần lớn đến từ tri thức ngôn ngữ tiền huấn luyện, vì với 8.012 mẫu thì học biểu diễn từ đầu là bất khả thi. Nhưng quy mô không thay cho chuyên biệt hóa: PhoBERT 135 triệu tham số ngang bằng XLM-R large 560 triệu tham số về accuracy ($p = 0,32$) và vượt hẳn về macro-F1 (0,466 so với 0,372).

CH3 — Tách từ tiếng Việt đáng giá bao nhiêu? 4,9 điểm accuracy. Đây là bước tiền xử lý bắt buộc chứ không phải tùy chọn cấu hình khi làm việc với PhoBERT.

CH4 — Mô hình sai ở đâu? Tập trung ở nhóm có độ trùng lặp từ vựng cao giữa hai câu, ở lớp *neutral* và *contradiction*, và mang thiên lệch hệ thống về phía kết luận *entailment*. Mức đồng thuận giữa các mô hình rất thấp (PhoBERT và XLM-R chỉ đồng ý ở 50,1% số mẫu) và 21,2% số mẫu bị cả bốn mô hình cùng dự đoán sai.

Mô hình được khuyến nghị cho bài toán này là PhoBERT: tốt nhất ở cả hai metric, đồng thời rẻ hơn hẳn ở mọi chiều chi phí. Cần nhấn mạnh rằng khuyến nghị này dựa trên macro-F1 và chi phí, không dựa trên accuracy — vì chênh lệch accuracy với XLM-R large không đạt ý nghĩa thống kê.

Cuối cùng, một bài học về phương pháp thực nghiệm. Vòng thí nghiệm đầu tiên của đồ án đã huấn luyện toàn bộ chín mô hình trên văn bản chưa tách từ do một cơ chế dự phòng im lặng, và sai sót này chỉ bị phát hiện nhờ đối chiếu chéo giữa các tập kết quả. Kinh nghiệm rút ra: mọi bước tiền xử lý có khả năng thất bại đều phải báo lỗi lớn tiếng thay vì âm thầm bỏ qua, và phải ghi lại trong tập kết quả rằng bước đó đã thực sự chạy hay chưa. Nếu thiếu điều kiện thứ hai, một sai sót có thể sống sót qua trọn một vòng thực nghiệm mà không để lại dấu vết nào.

8.2 Hướng phát triển

1. **ESIM hoặc BiLSTM có co-attention.** Đây là thí nghiệm có giá trị khoa học cao nhất trong danh sách, vì nó kiểm chứng trực tiếp luận điểm trung tâm của báo cáo. Nếu một mô hình RNN có cross-attention thu hẹp được đáng kể khoảng cách với PhoBERT, luận điểm “cross-attention quan trọng hơn quy mô” được xác nhận; nếu không, cần quy phần lớn khoảng cách cho pretraining.
2. **Nạp embedding tiền huấn luyện PhoW2V** cho TextCNN và BiLSTM, đồng thời khôi phục nhánh ablation về pretrained embedding đã bị bỏ.
3. **Ensemble PhoBERT và XLM-R large** bằng cách cộng logit. Hai mô hình chỉ đồng thuận ở 50,1% số mẫu và 78,8% số mẫu có ít nhất một mô hình đúng, nên dư địa cải thiện là có thật.
4. **Chạy nhiều seed và báo cáo trung bình kèm độ lệch chuẩn**, thay cho kết quả một seed hiện tại.
5. **Xử lý riêng lớp *contradiction*.** Đây là lớp yếu nhất ở ba trong bốn mô hình và cũng là lớp quan trọng nhất cho ứng dụng thực tế. Hướng khả thi: trọng số lớp, focal loss, hoặc tăng cường dữ liệu bằng cách sinh thêm cặp mâu thuẫn.

Tài liệu tham khảo

FBSB

Tài liệu tham khảo

- [1] T. V. Huynh, K. V. Nguyen, and N. L.-T. Nguyen, “A New Benchmark Dataset and Mixture-of-Experts Language Models for Adversarial Natural Language Inference in Vietnamese,” *Expert Systems with Applications*, p. 130109, 2025. doi: 10.1016/j.eswa.2025.130109.
- [2] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained Language Models for Vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037–1042.
- [3] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised Cross-lingual Representation Learning at Scale,” *arXiv preprint arXiv:1911.02116*, 2019.
- [4] Y. Kim, “Convolutional Neural Networks for Sentence Classification,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751.
- [5] Q. Chen, X. Zhu, Z. Ling, S. Wei, H. Jiang, and D. Inkpen, “Enhanced LSTM for Natural Language Inference,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017, pp. 1657–1668.
- [6] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, and A. Bordes, “Supervised Learning of Universal Sentence Representations from Natural Language Inference Data,” in *Proceedings of EMNLP*, 2017, pp. 670–680.
- [7] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

Checklist tái lập thực nghiệm

- ☐ Repository có README và cấu trúc thư mục rõ ràng.
- ☐ Có hướng dẫn tải/chuẩn bị dữ liệu.
- ☐ Có tệp khai báo thư viện và phiên bản.
- ☐ Có lệnh train, evaluate và inference.
- ☐ Có seed, cấu hình và checkpoint/model ID.
- ☐ Không commit dữ liệu nhạy cảm, token hoặc khóa API.
- ☐ Có license/citation cho mã nguồn và mô hình tham khảo.
- ☐ Kết quả trong báo cáo có thể tạo lại từ code.

A.1 Ví dụ cấu trúc repository

```
1 project/  
2 |-- README.md  
3 |-- requirements.txt  
4 |-- configs/  
5 |-- data/  
6 |-- notebooks/  
7 |-- src/  
8 |   |-- datasets.py  
9 |   |-- models.py  
10 |   |-- train.py  
11 |   |-- evaluate.py  
12 |-- outputs/  
13 |-- reports/
```

Listing A.1: Cấu trúc thư mục gợi ý

Danh sách chủ đề và bộ dữ liệu gợi ý

Hướng dẫn viết

Danh sách dưới đây tổng hợp các chủ đề do giảng viên cung cấp. Nhóm cần kiểm tra lại giấy phép, phiên bản và khả năng truy cập trước khi bắt đầu. Các nguồn chỉ có tên mà chưa có URL đầy đủ cần được xác nhận lại trên LMS hoặc với giảng viên.

Bảng B.1: Các chủ đề đồ án Deep Learning gợi ý

STT	Chủ đề	Nguồn dữ liệu
1	Nhận diện cảm xúc trong bình luận mạng xã hội tiếng Việt	https://huggingface.co/datasets/tridm/UIT-VSMEC
2	Phát hiện biển báo giao thông bằng YOLO/DETR nhẹ	<i>Traffic Signs Detection – liên kết do giảng viên/LMS cung cấp</i>
3	Phân loại rác thải bằng ảnh	https://www.kaggle.com/datasets/techsash/waste-classification-data
4	Visual Question Answering tiếng Việt	https://github.com/kh4nh12/ViVQA
5	Vietnamese Hate Speech Detection theo đối tượng	https://huggingface.co/datasets/sonlam1102/vithsd
6	Nhận diện bệnh lá cà chua	https://www.kaggle.com/datasets/kaustubhb999/tomatoleaf/data
7	Dự đoán nhóm bệnh từ mô tả triệu chứng tiếng Việt	https://www.kaggle.com/datasets/pb30025030/vimedical-disease
8	Question Answering tiếng Việt	https://huggingface.co/datasets/taidng/UIT-ViQuAD2.0

Tiếp tục ở trang sau

STT	Chủ đề	Nguồn dữ liệu
9	Phân loại rối loạn nhịp tim từ ECG 12 đạo trình	https://www.kaggle.com/datasets/erarayamorenzomuten/chapmanshaoxing-12lead-ecg-database/data
10	Phân loại tổn thương da bằng ảnh dermoscopy	https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000
11	Phát hiện phàn nàn trong đánh giá thương mại điện tử tiếng Việt	https://huggingface.co/datasets/tarudesu/ViOCD
12	Phân tích cảm xúc phản hồi sinh viên tiếng Việt theo sentiment và topic	https://huggingface.co/datasets/uitnlp/vietnamese_students_feedback
13	Phân loại rác thải bằng ảnh (RealWaste)	https://archive.ics.uci.edu/dataset/908/realwaste
14	Phân loại cảnh quan tự nhiên và đô thị từ ảnh	https://www.kaggle.com/datasets/puneet6060/intel-image-classification
15	Phát hiện vết nứt trên bề mặt bê tông	https://www.kaggle.com/datasets/arnavr10880/concrete-crack-images-for-classification
16	Suy luận ngôn ngữ tự nhiên tiếng Việt	https://huggingface.co/datasets/uitnlp/ViANLI
17	Phân loại ý định câu hỏi chăm sóc khách hàng ngân hàng	https://huggingface.co/datasets/PolyAI/banking77
18	Phân tích cảm xúc, hài hước, châm biếm và nội dung phản cảm trong meme	https://www.kaggle.com/datasets/williamscott701/memotion-dataset-7k
19	Sinh mô tả ảnh bằng tiếng Việt	https://huggingface.co/datasets/ThucPD/UIT-ViIC
20	Nhận diện ký tự ngôn ngữ ký hiệu từ ảnh bàn tay	https://www.kaggle.com/datasets/datumunge/sign-language-mnist

Tiếp tục ở trang sau

STT	Chủ đề	Nguồn dữ liệu
21	Nhận diện bệnh trên lá sắn bằng ảnh thực tế	https://www.kaggle.com/competitions/cassava-leaf-disease-classification/data
22	Hỏi đáp tiếng Việt dựa trên nội dung chữ trong ảnh	https://huggingface.co/datasets/minhquan6203/ViTextVQA
23	Nhận diện cảm xúc trong nội dung mạng xã hội tiếng Anh	https://huggingface.co/datasets/dair-ai/emotion
24	Active AI Camera – bảo đảm an toàn lao động ở nhà máy (nhận diện mũ bảo hộ/PPE)	https://universe.roboflow.com/mohamed-traore-2ekkp/ppe-detection-l80fg

Gợi ý ánh xạ mô hình theo loại dữ liệu

Bảng C.1: Các lựa chọn kiến trúc phù hợp để đáp ứng yêu cầu bốn mô hình

Loại dữ liệu/tác vụ	Gợi ý lựa chọn mô hình
Ảnh phân loại	CNN-based: CNN tự xây dựng, ResNet hoặc EfficientNet. RNN/LSTM-based: CNN-LSTM hoặc biểu diễn ảnh thành chuỗi patch khi có cơ sở. Transformer-based: ViT hoặc Swin Transformer. External: mô hình chuyên biệt từ paper/GitHub/Kaggle.
Văn bản	CNN-based: TextCNN hoặc Conv1D. RNN/LSTM-based: BiLSTM hoặc GRU. Transformer-based: BERT, PhoBERT hoặc Transformer encoder. External: mô hình/repository ngoài nội dung lab.
Chuỗi ECG	CNN-based: 1D-CNN. RNN/LSTM-based: LSTM hoặc GRU. Transformer-based: time-series Transformer. External: mô hình từ bài báo y sinh.
Object detection	CNN-based: YOLO hoặc Faster R-CNN. RNN/LSTM-based: kiến trúc hybrid/sequence chỉ khi có cơ sở và được giảng viên phê duyệt. Transformer-based: DETR hoặc Deformable DETR. External: detector khác hoặc repository công khai.
Đa phương thức	CNN-based: CNN encoder cho ảnh. RNN/LSTM-based: RNN decoder hoặc cơ chế fusion tuần tự. Transformer-based: vision-language Transformer. External: mô hình VQA/image captioning từ paper.

Checklist tự đánh giá trước khi nộp

- ☐ Báo cáo phát biểu rõ bài toán, đầu vào, đầu ra và mục tiêu.
- ☐ Phân tích dữ liệu đủ sâu và không có data leakage.
- ☐ Có tối thiểu ba mô hình thuộc nội dung CNN, RNN/LSTM và Transformer.
- ☐ Có một mô hình tham khảo bên ngoài và trích dẫn đúng nguồn.
- ☐ Giao thức đánh giá công bằng, metric phù hợp và test set độc lập.
- ☐ Có bảng kết quả, learning curve, phân tích theo lớp và error analysis.
- ☐ Có thảo luận về chi phí tính toán, hạn chế và đạo đức.
- ☐ Mã nguồn có thể chạy lại và kết quả khớp với báo cáo.
- ☐ Đã kiểm tra chính tả, hình/bảng, số trang và liên kết.